PYTHON FUNDAMENTALS

Conditional Statements

50-question practice workbook

Easy 15 | Medium 15 | Hard 15 | Real-world 5

15 EASY | 15 MEDIUM | 15 HARD | 5 PRACTICAL

What you will practise

- if / elif / else and correctly ordered branches
- Comparison operators: ==, !=, <, <=, >, >=
- Boolean logic: and, or, not; compound conditions
- Boundary cases, invalid inputs, and business rules

How to use this workbook

1. Read the requirement and plan the conditions before coding.
2. Solve in VS Code without searching for a complete solution.
3. Test the sample case plus boundary and invalid inputs.
4. Mark your result in the tracker after you can explain the logic.

No solution code is included intentionally. The goal is independent problem solving.

Scope: Python fundamentals only. No DSA, external libraries, or loop-based solutions are required.

Easy

Build confidence with one clear condition at a time.

E01 Positive, negative, or zero

Read an integer and classify its sign.

Requirements: Print exactly one of: Positive, Negative, or Zero.

Test: Input: -4 | Output: Negative

My logic:

E02 Even or odd

Read an integer and decide whether it is even or odd.

Requirements: Print Even or Odd. Test positive numbers, negative numbers, and zero.

Test: Input: 12 | Output: Even

My logic:

E03 Divisible by five

Read an integer and determine whether it is divisible by 5.

Requirements: Print Divisible or Not divisible. A number is divisible when the remainder is zero.

Test: Input: 35 | Output: Divisible

My logic:

E04 Compare two numbers

Read two integers and compare them.

Requirements: Print First is greater, Second is greater, or Both are equal.

Test: Input: 8, 8 | Output: Both are equal

My logic:

E05 Voting age check

Read a person's age in whole years. Decide whether the person meets the voting-age threshold used in this exercise.

Requirements: Use 18 as the minimum age. Assume the age is not negative.

Test: Input: 18 | Output: Eligible to vote

My logic:

Easy

Continue the same rules: define every condition clearly and test boundary cases.

E06 Pass or fail

Read a test score from 0 to 100.

Requirements: A score of 40 or higher passes. Print Pass or Fail.

Test: Input: 39 | Output: Fail

My logic:

E07 Assign a grade

Read a whole-number score from 0 to 100 and assign a letter grade.

Requirements: A: 90-100; B: 80-89; C: 70-79; D: 60-69; F: below 60.

Test: Input: 84 | Output: B

My logic:

E08 Weekend or weekday

Read a day number, where Monday is 1 and Sunday is 7.

Requirements: Print Weekday for 1-5, Weekend for 6-7, and Invalid day for any other number.

Test: Input: 7 | Output: Weekend

My logic:

E09 Inclusive range check

Read an integer and check whether it falls from 10 through 50, including both endpoints.

Requirements: Print Inside range or Outside range.

Test: Input: 10 | Output: Inside range

My logic:

E10 Vowel or consonant

Read one English alphabet letter. The input will contain exactly one letter.

Requirements: Treat uppercase and lowercase letters the same. Print Vowel or Consonant.

Test: Input: E | Output: Vowel

My logic:

Easy

Continue the same rules: define every condition clearly and test boundary cases.

E11 Divisibility by 3 and 7

Read an integer and check divisibility by both 3 and 7.

Requirements: Print Both when divisible by both, 3 only, 7 only, or Neither.

Test: Input: 21 | Output: Both

My logic:

E12 Temperature label

Read a whole-number temperature in degrees Celsius.

Requirements: Below 0: Freezing; 0-15: Cold; 16-25: Mild; 26-35: Warm; above 35: Hot.

Test: Input: 25 | Output: Mild

My logic:

E13 Simple login check

Ask for a username and password. The expected username is student and password is learn123.

Requirements: Both values must match exactly. Print Login successful or Invalid credentials.

Test: Username: student, password: learn123 | Output: Login successful

My logic:

E14 Three-digit positive number

Read an integer and check whether it is a positive three-digit number.

Requirements: Accept values from 100 through 999 only.

Test: Input: 99 | Output: Not a three-digit positive number

My logic:

E15 Is one number a factor of another?

Read integers a and b. Decide whether b is a non-zero factor of a.

Requirements: Do not attempt modulo by zero. Print Factor or Not a factor.

Test: Input: a=20, b=0 | Output: Not a factor

My logic:

Medium

Combine conditions and make boundary decisions carefully.

M01 Largest of three numbers

Read three integers and print the largest value.

Requirements: Your logic must still work when two or all three values are equal. Do not use max().

Test: Input: 9, 4, 9 | Output: 9

My logic:

M02 Validate and classify a triangle

Read three positive side lengths. First decide whether they can form a triangle; then classify a valid triangle.

Requirements: A valid triangle has each pair of sides summing to more than the remaining side. Classify as Equilateral, Isosceles, or Scalene.

Test: Input: 3, 4, 5 | Output: Scalene

My logic:

M03 Leap year checker

Read a year and decide whether it is a leap year under the Gregorian calendar rule.

Requirements: A year divisible by 400 is leap; otherwise a year divisible by 100 is not; otherwise a year divisible by 4 is leap.

Test: Input: 1900 | Output: Not a leap year

My logic:

M04 Classify a character

Read a single character.

Requirements: Print Vowel, Consonant, Digit, or Other. Handle uppercase and lowercase letters.

Test: Input: 7 | Output: Digit

My logic:

M05 BMI category

Read weight in kilograms and height in metres, then calculate BMI = weight / height squared.

Requirements: Reject non-positive weight or height. For valid inputs: below 18.5 Underweight; below 25 Normal; below 30 Overweight; otherwise Obesity.

Test: Input: 70, 1.75 | Output: Normal

My logic:

Medium

Continue the same rules: define every condition clearly and test boundary cases.

M06 Age-based ticket price

Read a visitor's age in whole years.

Requirements: Negative ages are invalid. Under 5: Free; 5-12: Child ticket INR 100; 13-59: Adult ticket INR 250; 60 and over: Senior ticket INR 150.

Test: Input: 65 | Output: Senior ticket - INR 150

My logic:

M07 Free shipping eligibility

Read an order total and whether the customer is a member (yes/no).

Requirements: Shipping is free when the order total is at least INR 1000, or when the customer is a member and the order total is at least INR 500. Otherwise shipping costs INR 80. Reject negative totals.

Test: Input: total=600, member=yes | Output: Free shipping

My logic:

M08 Basic calculator

Read two numbers and an operator chosen from +, -, *, and /.

Requirements: Perform the requested operation. Reject unknown operators and prevent division by zero.

Test: Input: 10, 0, / | Output: Cannot divide by zero

My logic:

M09 Tiered discount

Read an order subtotal in INR and calculate its discount and final amount.

Requirements: Subtotal >= 5000 gets 20%; >= 2500 gets 10%; otherwise no discount. Reject negative subtotals. Apply only one tier.

Test: Input: 3000 | Output: Discount INR 300; final INR 2700

My logic:

M10 Rock, paper, scissors

Read two players' choices: rock, paper, or scissors.

Requirements: Reject any invalid choice. Otherwise print Draw or identify the winner. Input is lowercase.

Test: Input: rock, scissors | Output: Player 1 wins

My logic:

Medium

Continue the same rules: define every condition clearly and test boundary cases.

M11 Classify a month into a quarter

Read a month number from 1 to 12.

Requirements: Print Q1 for months 1-3, Q2 for 4-6, Q3 for 7-9, Q4 for 10-12, and Invalid month otherwise.

Test: Input: 10 | Output: Q4

My logic:

M12 Dashboard access by role

Read whether the user is logged in (yes/no) and whether the user is an admin (yes/no).

Requirements: If not logged in, print Please log in. If logged in and admin, print Admin dashboard. Otherwise print User dashboard.

Test: Input: logged in=yes, admin=no | Output: User dashboard

My logic:

M13 Simple password rule

Read a password string.

Requirements: Accept it only if it has at least 8 characters and contains no spaces. Print Accepted or Rejected. Do not check special-character or digit rules.

Test: Input: blue sky | Output: Rejected

My logic:

M14 Number size band

Read an integer and place it into a size band.

Requirements: Negative: Negative; 0-10 inclusive: Small; 11-100 inclusive: Medium; above 100: Large.

Test: Input: 100 | Output: Medium

My logic:

M15 Electricity bill with slabs

Read non-negative electricity usage in whole units. Calculate the total energy charge.

Requirements: First 100 units cost INR 5 per unit; units 101-300 cost INR 7 each; units above 300 cost INR 10 each. This is progressive billing: each rate applies only to units in its slab.

Test: Input: 150 | Output: INR 850

My logic:

Hard

Stress-test compound logic, validation, and edge cases.

H01 Validated grading system

Read a score and return a grade.

Requirements: First reject scores outside 0-100. For valid scores use A: 90-100, B: 80-89, C: 70-79, D: 60-69, F: below 60. Also distinguish Invalid score from a valid F.

Test: Input: 100 | Output: A

My logic:

H02 Scholarship eligibility

Read GPA (0.0-4.0), annual family income in INR, and attendance percentage (0-100).

Requirements: Eligible if GPA is at least 3.7; OR GPA is at least 3.5 AND income is at most INR 300000 AND attendance is at least 90%. Reject values outside the stated ranges.

Test: Input: GPA=3.6, income=250000, attendance=92 | Output: Eligible

My logic:

H03 Progressive income tax

Read a non-negative annual taxable income in INR and calculate tax using progressive slabs.

Requirements: First INR 250000: 0%; next INR 250000: 5%; next INR 500000: 10%; any amount above INR 1000000: 20%. Each rate applies only to income inside its slab.

Test: Input: 600000 | Output: INR 22500

My logic:

H04 Validate a calendar date

Read day, month, and year; decide whether the date exists in the Gregorian calendar.

Requirements: Handle month lengths, February, leap years, invalid months, and impossible days. Use conditional statements; you may assume all three inputs are integers.

Test: Input: day=29, month=2, year=2024 | Output: Valid date

My logic:

H05 ATM withdrawal decision

Read withdrawal amount and available balance in INR.

Requirements: Apply checks in this order: amount must be positive; amount must be a multiple of 100; amount must not exceed INR 20000; balance must cover amount plus a INR 20 fee. Print the first applicable rejection reason, or print withdrawal approved and remaining balance.

Test: Input: amount=500, balance=510 | Output: Insufficient balance for fee

My logic:

Hard

Continue the same rules: define every condition clearly and test boundary cases.

H06 Parking charge with partial hours

Read parking duration as a positive number of hours. Any started hour counts as a full hour.

Requirements: Charge INR 40 for the first hour and INR 20 for each additional billed hour. Cap the total at INR 200. Reject zero or negative duration.

Test: Input: 1.2 hours | Output: INR 60

My logic:

H07 Locate a point on a plane

Read x and y coordinates as numbers.

Requirements: Print Origin if both are zero; X-axis if only y is zero; Y-axis if only x is zero; otherwise print Quadrant I, II, III, or IV according to the coordinate signs.

Test: Input: x=-3, y=4 | Output: Quadrant II

My logic:

H08 Loan pre-screening

Read applicant age, monthly income in INR, and credit score.

Requirements: Reject invalid age (below 18) or score (outside 300-850). Pre-screen as Eligible only if age is 21-60 inclusive, income is at least INR 30000, and score is at least 700. Otherwise print Not eligible.

Test: Input: age=25, income=35000, score=710 | Output: Eligible

My logic:

H09 Delivery fee by zone and order value

Read zone (local/extended), order total in INR, and whether the order contains a fragile item (yes/no).

Requirements: Local delivery costs INR 30, extended costs INR 70. Orders of at least INR 1500 get free base delivery. Fragile items add INR 25 even when base delivery is free. Reject unknown zones or negative totals.

Test: Input: local, INR 1600, fragile=yes | Output: INR 25

My logic:

H10 Support-ticket priority

Read impact (one user/all users), severity (low/medium/high/critical), and whether a core service is down (yes/no).

Requirements: P1 if all users are affected and severity is high or critical, OR a core service is down. Otherwise P2 for high/critical severity, P3 for medium, and P4 for low. Reject invalid labels.

Test: Input: all users, critical, core service down=no | Output: P1

My logic:

Hard

Continue the same rules: define every condition clearly and test boundary cases.

H11 Validate time and label the daypart

Read hour (0-23) and minute (0-59).

Requirements: Reject invalid times. For valid times: Night 00:00-05:59; Morning 06:00-11:59; Afternoon 12:00-16:59; Evening 17:00-23:59.

Test: Input: 17, 0 | Output: Evening

My logic:

H12 Mobile data overage bill

Read the plan allowance and actual usage in GB. Both inputs are non-negative.

Requirements: If usage is within allowance, overage is INR 0. For the first 5 GB above allowance charge INR 20 per GB; for any additional overage above 5 GB charge INR 30 per GB. Use proportional charges for fractional GB.

Test: Input: allowance=10, usage=17 | Output: INR 160

My logic:

H13 Driver insurance premium

Read base premium in INR, driver age, number of at-fault claims in the past 3 years, and defensive-driving-course status (yes/no).

Requirements: Add 25% of base premium if age is under 25; add 15% of base premium if claims are 2 or more; then subtract 10% of the original base premium if the course is completed. Reject negative base premium, age, or claim count.

Test: Input: base=10000, age=22, claims=2, course=yes | Output: INR 13000

My logic:

H14 Sort three values using conditions

Read three distinct integers and print them in ascending order, without using sort(), min(), or max().

Requirements: Use comparisons and conditional branches to determine the smallest, middle, and largest values. The inputs are guaranteed to be distinct.

Test: Input: 8, 2, 5 | Output: 2, 5, 8

My logic:

H15 Order promotion rules

Read subtotal in INR, membership status (yes/no), and coupon code.

Requirements: Apply exactly one discount: code SAVE20 gives 20% off only for members with subtotal >= INR 2000; otherwise, members get 5% off for subtotal >= INR 1000; otherwise no discount. Unknown coupon codes do not invalidate the order; report that the coupon was not applied. Reject negative subtotals.

Test: Input: subtotal=2500, member=yes, code=SAVE20 | Output: Discount INR 500; final INR 2000

My logic:

Real-World Challenges

Turn business rules into precise conditional logic.

R01 E-commerce checkout summary

Build a checkout calculator for an online store. Inputs: subtotal in INR, membership status (yes/no), and coupon code.

Requirements: Member discount is 5% when subtotal >= INR 1000. Coupon SAVE10 gives 10% off when subtotal >= INR 1500. Only the larger single discount applies; discounts do not stack. After discount, shipping is free if the payable subtotal is >= INR 500, otherwise shipping is INR 60. Reject negative subtotal. Print discount, shipping, and final total.

Test: Input: subtotal=1600, member=yes, code=SAVE10 | Expected: discount INR 160; shipping INR 0; total INR 1440

My logic:

R02 Employee performance bonus

A company calculates an annual bonus from salary, rating (1-5), and years at the company.

Requirements: Rating 5: 15% of salary; rating 4: 10%; rating 3: 5%; rating 1-2: 0%. Employees with at least 5 years receive an additional 2% of salary, added to the bonus. Reject negative salary or years and ratings outside 1-5. Print the bonus amount.

Test: Input: salary=600000, rating=4, years=6 | Expected: INR 72000

My logic:

R03 ATM cash withdrawal with account rules

Create a decision flow for an ATM withdrawal. Inputs: amount requested, balance, and whether the account is active (yes/no).

Requirements: Reject inactive accounts first, then non-positive amounts, then amounts not divisible by 100, then withdrawals above INR 20000, then requests that leave less than INR 500 in the account after a INR 20 fee. If approved, print the cash amount and remaining balance after fee.

Test: Input: amount=1000, balance=2000, active=yes | Expected: Approved; remaining balance INR 980

My logic:

Real-World Challenges (Continued)

Continue the same rules: define every condition clearly and test boundary cases.

R04 Food delivery fee and eligibility

A food delivery service uses order value, distance in kilometres, and weather status (normal/severe).

Requirements: Orders over 10 km are unavailable. Severe weather makes delivery unavailable beyond 5 km. Base fee is INR 30 for distances up to 3 km, INR 50 for distances above 3 and up to 7 km, and INR 80 for distances above 7 and up to 10 km. Orders of at least INR 800 get a INR 30 fee discount, but the final fee cannot be negative. Reject negative distance or order value and invalid weather labels.

Test: Input: order=900, distance=6, weather=normal | Expected: Delivery available; fee INR 20

My logic:

R05 University admission pre-check

A college pre-checks applicants using entrance score (0-100), high-school percentage (0-100), and an interview result (pass/fail).

Requirements: Reject invalid numeric values. First check regular admission: score >= 80 AND high-school percentage >= 75 AND interview is pass. If that rule fails, offer Conditional admission when score >= 90 AND interview is pass. Otherwise print Not admitted. Check regular admission first.

Test: Input: score=92, percentage=68, interview=pass | Expected: Conditional admission

My logic:

Practice Tracker

Mark each question only when you can solve it and explain why your conditions work. Use R for revised, H for solved with a hint, and I for independent.

Question	Status	Date
[ ] E01	_____	_____
[ ] E02	_____	_____
[ ] E03	_____	_____
[ ] E04	_____	_____
[ ] E05	_____	_____
[ ] E06	_____	_____
[ ] E07	_____	_____
[ ] E08	_____	_____
[ ] E09	_____	_____
[ ] E10	_____	_____
[ ] E11	_____	_____
[ ] E12	_____	_____
[ ] E13	_____	_____
[ ] E14	_____	_____
[ ] E15	_____	_____
[ ] M01	_____	_____
[ ] M02	_____	_____
[ ] M03	_____	_____
[ ] M04	_____	_____
[ ] M05	_____	_____
[ ] M06	_____	_____
[ ] M07	_____	_____
[ ] M08	_____	_____
[ ] M09	_____	_____
[ ] M10	_____	_____

Question	Status	Date
[ ] M11	_____	_____
[ ] M12	_____	_____
[ ] M13	_____	_____
[ ] M14	_____	_____
[ ] M15	_____	_____
[ ] H01	_____	_____
[ ] H02	_____	_____
[ ] H03	_____	_____
[ ] H04	_____	_____
[ ] H05	_____	_____
[ ] H06	_____	_____
[ ] H07	_____	_____
[ ] H08	_____	_____
[ ] H09	_____	_____
[ ] H10	_____	_____
[ ] H11	_____	_____
[ ] H12	_____	_____
[ ] H13	_____	_____
[ ] H14	_____	_____
[ ] H15	_____	_____
[ ] R01	_____	_____
[ ] R02	_____	_____
[ ] R03	_____	_____
[ ] R04	_____	_____
[ ] R05	_____	_____

Quick reference

Branching: **if** checks the first condition; **elif** checks another condition only if previous branches failed; **else** handles everything left. Conditions are checked in order.

Comparisons: == equal, != not equal, < less than, <= less than or equal, > greater than, >= greater than or equal.

Boolean logic: and requires both conditions to be true; or requires at least one; not reverses a truth value. Use parentheses when they make a compound condition easier to read.

Official references: Python Tutorial - if statements: https://docs.python.org/3/tutorial/controlflow.html ; Built-in Types - truth values, Boolean operations, and comparisons: https://docs.python.org/3/library/stdtypes.html

Workbook note: The questions are original practice exercises informed by the official Python documentation. Real-world examples use simplified rules for programming practice, not real financial, legal, medical, or admissions advice.